Dentacore Authentication Design: Sessions, Redis & 2FA

Nov 4, 2026 · @Nam

Summary

Dentacore keeps server-side Redis sessions (opaque session ID in a cookie) instead of access/refresh JWTs, adds idle + absolute timeouts, and requires TOTP 2FA for staff. Auth is built in-house, not with the official NestJS authentication package.

Area	Decision
Session model	Random opaque session ID in an HttpOnly cookie, verified against Redis on every request
Why not JWT	Instant revocation would need a per-request denylist lookup anyway; extra complexity for no gain on a single API
Redis topology	Two instances: auth (`REDIS_AUTH`: sessions + all short-lived auth state; noeviction, AOF on, HA) and cache (allkeys-lru, no persistence)
Extension	Extend TTL on every request; no throttled-extension trick
Timeouts	Idle 30 min (sliding), absolute 12 h (timestamp stored in session)
Expiry UX	Input-triggered extend, server-side autosave, re-login modal with request replay
Remember me	Longer session class, MFA required, 7 d idle / 30 d absolute, cookie `Max-Age` = 30 d
Login identifier	Employee number (unique, stable); email only for recovery
2FA	TOTP (otplib v13), mandatory for staff; MFA challenge in Redis (5 min); replay protection via `lastUsedTimeStep`

Area	Decision
Storage	Invite token SHA-256 hashed; TOTP secret encrypted with XChaCha20-Poly1305 (`@noble/ciphers`), employee ID as AAD
Concurrency	Every once-only transition is a conditional update (compare-and-set)

Session model: Redis sessions, not JWTs

We keep the opaque session ID in a cookie, checked against Redis on every request. That gives instant admin revocation and session tracking.

Why not access/refresh JWTs

- Instant revocation with JWTs needs a denylist or per-user session version checked on every request. That is a per-request lookup again.
- JWTs add a refresh flow, token rotation, reuse detection and client token handling.
- Healthcare data needs revocation to take effect immediately (e.g. a staff member is let go), not when a 15-minute token expires.

If we ever move to JWTs, use the hybrid pattern

- Short-lived access JWTs (5–10 min), verified statelessly.
- Refresh tokens stored server-side as the session record (session list, per-device logout).
- Revocation deletes the refresh record; for instant cutoff, add the access token's `jti` to a denylist with TTL = remaining lifetime, broadcast to instances.

This only pays off when many independent services must verify identity without a central store. A single NestJS API doesn't need it.

Build vs the official package: we build auth in-house. The NestJS authentication package follows the same design (hashed `__Host-sid` sessions, AES-256-GCM TOTP secrets, pending session before MFA, staged secret `replace`), but makes 2FA opt-in per user, keeps sessions in Postgres via Drizzle/TypeORM with no Redis mentioned, and defaults to 14-day absolute / 3-day idle sessions. Its `replace` flow is a useful reference for secret regeneration.

Redis capacity and scaling

One Redis node handles 100k concurrent users comfortably; availability is the real concern. All figures below are estimates to confirm by load testing.

Measure	Estimate at 100k concurrent users
Auth lookups	10–20k/sec (one request per 5–10 s); ~50k/sec in a heavy burst
Single-node capacity	~100–200k simple GET/GETEX ops/sec
Memory	100k sessions × ~0.5 KB ≈ 50 MB
Network	~20 MB/s at 20k ops/sec

What matters at that scale

1. **High availability.** Primary + replica with failover (Sentinel or a managed service). AOF persistence so sessions survive restarts.
2. **Node client.** One shared client per app instance; enable auto-pipelining (`enableAutoPipelining` in ioredis).
3. **O(1) commands only.** No `KEYS *` or large values. Keep a per-user set of session IDs for listing sessions.
4. App servers and Postgres need horizontal scaling before Redis does.
5. Beyond one node, Redis Cluster scales roughly linearly; session keys shard cleanly.

Throttled-extension trick: not used

- The trick: only EXPIRE when remaining TTL < IDLE/2, pipelined GET + TTL otherwise.
- It saves write traffic (replication, AOF), not commands: 2 commands per request vs 1 for GETEX. Worse on per-command billing.
- It makes the idle timeout fuzzy (between half and the full value).
- Only worth it near single-node limits (~50k+ session ops/sec, sustained Redis CPU > 60–70%, or replication lag).

Verify: `redis-benchmark -t get,set -d 512 -c 100` on the target instance, then k6 against the NestJS auth guard.

Separate Redis instances: sessions vs cache

Sessions and cache need opposite eviction and persistence settings, and both are per instance, so they get separate instances before production.

	Sessions	Cache
When memory is full	Must never be evicted	Should be evicted
`maxmemory-policy`	`noeviction`	`allkeys-lru` / `allkeys-lfu`
Persistence	AOF on	Off
If it goes down	Everyone locked out	App slower, still works

Why one instance fails

- `allkeys-lru`: cache bursts evict active sessions, causing random logouts.
- `noeviction`: a full cache makes writes fail, so logins fail.
- `volatile-lru` doesn't help; sessions have TTLs too.
- Logical databases (`SELECT 1`) share memory and policy, aren't supported in Cluster, and `FLUSHALL` wipes both.

In NestJS: two clients (`REDIS_AUTH_URL`, `REDIS_CACHE_URL`) injected under different tokens; `@nestjs/cache-manager` points at the cache instance only. Sharing one instance is fine for local development only.

Sessions instance keyspace

All short-lived auth state lives in the sessions instance, not the cache: the cache's eviction could drop a pending activation mid-setup, the cache should be flushable at any time, and these keys hold password hashes and encrypted TOTP secrets. They share the sessions instance's needs, so no third instance.

Key	Contents	TTL
`session:{hash}`	Session payload	Idle timeout, capped at absolute
`user:{id}:sessions`	Set of the user's session hashes	None; remove stale entries when a session is deleted or found expired
`mfa:{id}`	Login MFA challenge	~5 min

Key	Contents	TTL
`activation:{tokenHash}`	Pending activation setup	~15 min

With `noeviction`, Redis never cleans up for you: set every temporary key's TTL in the creating command (`SET activation:{tokenHash} <value> EX 900`), never SET then EXPIRE. Keys without a TTL accumulate until memory fills and logins fail.

Per-user session set

- Members are session ID **hashes**, never raw IDs (a raw ID is a working credential).
- Set TTL only ever lengthens: `EXPIRE key ttl NX` then `EXPIRE key ttl GT` in the same `MULTI` (GT alone never sets a TTL on a new key). A short session can't cut off a longer remember-me session's tracking.
- Stale hashes are pruned at login: `SMEMBERS` → pipelined `EXISTS` per hash → `SREM` the ones that returned 0 (errors are kept). Also prune when listing or revoking. No background sweeper and no keyspace notifications.
- Don't expose the hash to the frontend as a session reference; use a separate random `sessionRef` or check ownership before revoking.

Units: Redis `EX` and `RedisTtl` are seconds; `Date.now()`, Express cookie `maxAge` and cache-manager TTLs are milliseconds.

Session security checklist

The opaque-ID cookie session is OWASP's recommended model; its security depends on the details below.

Required

- [ ] Session ID from a CSPRNG: `crypto.randomBytes(32).toString('base64url')` (256 bits)
- [ ] New session ID on login and on privilege change (role or password change)
- [ ] Logout deletes the session from Redis, not just the cookie
- [ ] Cookie: `HttpOnly`, `Secure`, `SameSite=Lax`, `Path=/`, named `__Host-sid`, no `Domain`
- [ ] Idle timeout (sliding) + absolute timeout (`absoluteExpiresAt` in the session)
- [ ] CSRF: verify `Origin` against an allowlist on POST/PUT/PATCH/DELETE (SameSite misses sibling subdomains)

- [ ] Per-user set `user:{id}:sessions` for listing and revoking sessions
- [ ] Revoke all sessions on password change/reset
- [ ] Update or revoke sessions when an admin changes a user's role
- [ ] HTTPS + HSTS; CORS `credentials: true` only for an explicit allowlist
- [ ] Never log session IDs; rate-limit login; Redis on a private network with ACL

Cheap and worth doing

- Store `session:{sha256(id)}`, not the raw ID, so a Redis leak yields no usable cookies.
- Store IP and user agent for display only (device list).
- Strict Content Security Policy against XSS.

Over-engineering (don't)

- Binding the session to an IP address (random logouts on mobile and clinic networks).
- User-agent or fingerprint checks.
- Rotating the session ID on every request (races between concurrent requests).
- Signing or encrypting a cookie that already holds a 256-bit random ID.
- Encrypting session payloads in Redis once keys are hashed.
- Stacking several CSRF defenses; SameSite + Origin check is enough for a JSON API.

Rule of thumb: if it breaks things for legitimate users, or you can't name the attack it stops, it's over-engineering. Higher-value next steps: MFA for staff and audit logging of patient-record access.

Timeouts: idle and absolute

Each session has two clocks: idle 30 min (sliding) and absolute 12 h (fixed); whichever runs out first ends it.

Clock	Behaviour	Protects against	Stored in
Idle	Pushed forward by every authenticated request	Walk-away on a shared clinic computer	Redis key TTL

Clock	Behaviour	Protects against	Stored in
Absolute	Never moves	Long-lived stolen cookies; forces periodic re-check of MFA, password, account status	`absoluteExpiresAt` in the session payload

Sliding-only is the common mistake: an active user or a polling tab keeps the session alive forever.

Per-request logic

```
session = GET session:{hash}
if !session → 401
remaining_absolute = absoluteExpiresAt - now
if remaining_absolute <= 0 → DEL, 401
EXPIRE session:{hash} min(IDLE_TTL, remaining_absolute)
```

Capping the TTL at the remaining absolute time makes Redis delete the key exactly at the hard limit.

Values: idle 30 min, absolute 12 h (one shift). Make them configurable per clinic later only if asked. A future patient portal can use more relaxed limits.

Session expiry UX

A user who types for longer than the idle timeout without any request gets a 401 on submit; we prevent that with input-triggered extension and autosave, and recover with a re-login modal that replays the request.

Why the 401 happens

1. The dentist opens the notes form: the last request the server sees.
2. Typing sends nothing to the backend.
3. At 30 min the Redis key expires and is deleted.
4. Submit at 40 min: `GET` returns nil → 401. Sliding expiry can't revive a deleted session, and the absolute timeout lives inside that deleted session, so nothing recreates it. Only a login creates a session.

Prevention

- **Input-triggered extension:** throttled extend request (at most every 5 min) fired by real input events, never by a timer. Details below.
- **Server-side autosave** of drafts every 1–2 min while content changes; doubles as the activity signal. Details below.
- **Expiry warning** ("logged out in 2 min — stay signed in?") for long reading without typing; expose remaining time via a response header.
- **Background requests don't extend:** polling, heartbeats and dashboards authenticate but skip sliding (e.g. a `@NoSlide()` decorator).
- **No `localStorage` / `sessionStorage` for drafts:** clinical notes on shared computers would be readable by the next user.
- Multiple tabs: share expiry state via `BroadcastChannel` so the warning shows in the active tab.

How the two activity mechanisms work

Both run in the frontend so the server sees a request whenever the user is genuinely active; typing alone is invisible to the server. "At most every 5 min" is a throttle (fires on the first input, then at most once per interval), not a debounce (which waits for input to stop).

Input-triggered extension

- Keep `lastServerTouchAt` in memory; the axios response interceptor updates it on every successful response, so ordinary API calls count.
- Passive listeners on `keydown`, `input`, `pointerdown`, `wheel` call:

```
function onUserActivity() {
  if (Date.now() - lastServerTouchAt > 5 * 60_000) {
    lastServerTouchAt = Date.now();   // set first, so a burst of keys sends
one request
    api.post('/auth/session/extend'); // response interceptor updates it
again
  }
}
```

- `POST /auth/session/extend` has no logic of its own: the global guard extends the session; the endpoint returns `204` plus the expiry header. It must not carry `@NoSlide()`.
- With a 30-min idle timeout, the worst case leaves at least 25 min; an active user costs at most one extra request per 5 min, usually none.

- Never a `setInterval` ping: it keeps an unattended screen logged in.

Server-side autosave

- Any edit sets `dirty = true`. Save at most every 60–120 s while dirty, plus immediately on `blur`, `visibilitychange` to hidden, and before the final submit. A successful save sets `dirty = false`.
- The save loop is a timer gated on user edits, so an unattended screen stops saving after one final save and still times out.
- `PUT /drafts/:id { content, version }` with optimistic concurrency: `409` on version mismatch, so two tabs can't silently overwrite each other.
- `Draft { id, userId, patientId, kind, content, version, updatedAt }`, deleted on final submit; patient data, so the same access rules and audit logging apply.

How they combine

- The autosave `PUT` is an authenticated request, so it extends the session and updates `lastServerTouchAt`. While writing notes, autosave alone keeps the session alive.
- The extend call covers activity without a save: clicking through a chart, choosing options, scrolling a patient's history.
- A `401` from either (e.g. after the 12 h absolute limit) goes through the interceptor queue and re-login modal; the draft is still in memory and the last autosave is on the server.

Recovery: re-login modal with replay

1. Submit returns 401.
2. Form state stays in memory (component / form library / store state); a login modal opens over the page. Never redirect to `/login` (it unmounts the form).
3. The modal shows the locked username ("Enter the password for dr.lan@clinic") with a "Sign in as someone else" link that does a full logout + reload.
4. After login, the frontend compares the new session's user ID with the one loaded at page start. Same user → replay; different user → discard queue and form state, reload.
5. The server attributes the replayed request to the new session's user and authorizes it as usual, so the frontend check prevents accidental misattribution but is not a security boundary.

Request queue (frontend, in memory)

Not RabbitMQ, and not a browser feature: browsers, `fetch` and axios don't retry 401s. Implement it in an axios response interceptor (~30–50 lines):

1. On 401, push `{ config, resolve, reject }` onto an array.
2. Open the modal once, however many requests fail together.
3. On same-user re-login, replay all and resolve the original promises.
4. On cancel, failure or different user, reject all and clear.

Details: exclude the login endpoint from the interceptor (wrong password is also 401); mark replayed requests (`_retried`) and reject on a second 401; never send `WWW-Authenticate` from the backend (triggers the browser's Basic-auth popup).

Remember me

We keep "remember me" and implement it as a longer session class (Option 1), not a separate remember-me token.

	Option 1: longer session class (chosen)	Option 2: separate remember-me token
How	Same Redis session, `remember: true` , longer TTLs, cookie with `Max-Age`	Long-lived hashed token in its own cookie; creates a fresh session when the normal one expires, then rotates
Revocation	Existing paths unchanged	Second credential to track and revoke
Risk	None extra	Concurrent tabs trip rotation reuse detection, causing false logouts

Without "remember me" the cookie has no `Max-Age` (browser-session cookie). Browsers may restore these, so server-side timeouts remain the real control.

Remember-me sessions slide like normal ones: each request resets the idle TTL to 7 days, capped by the 30-day absolute limit (`min(idle, remaining absolute)`). Set the cookie's `Max-Age` to the absolute limit (30 days); a shorter value logs users out early, a longer one only sends a dead ID.

Safeguards for staff

- Require MFA to create a remember-me session.

- Limits: 7 days idle, 30 days absolute.
- Checkbox label: "Keep me signed in — don't use on shared computers."
- Per-clinic setting to disable it.
- Mark remember-me sessions in the session list.

Related, lower-risk features: remember the username (prefill), and "remember this device for MFA" for 30 days (still requires the password and normal timeouts).

2FA onboarding: activation flow

An admin invites the user by email; the user sets a password and TOTP, and nothing is committed until the first OTP verifies.

Rules

- Activation token: `crypto.randomBytes(32)`, stored hashed, single-use, expires in 24–72 h. Re-inviting revokes earlier tokens.
- GET is read-only. Email scanners (Defender Safe Links, Mimecast, Proofpoint) and link previews open links automatically, some running JavaScript. Only a POST carrying human input (password, OTP) may consume the token. No auto-submit on page load.
- One TOTP secret per setup. After step 2 it exists in Redis (pending, encrypted) and in the user's authenticator app; no `TotpCredential` row exists yet.

Steps

1. **Open link: validate only.** `GET /auth/activation?token=…` hashes and looks up the token, checks expiry/used, returns `valid` + email to display. Changes nothing.
2. **Submit password: prepare.** `POST /auth/activation/setup { token, password }` validates token and password rules, generates the pending TOTP secret, and stores a pending setup in Redis keyed by `tokenHash` (TTL ~15 min): `{ tokenHash, passwordHash, encryptedSecret }`. Returns the QR / `otpauth://` URI. Token not consumed; restarting replaces the pending setup (tell the user to rescan and delete the old entry).
3. **Submit OTP: commit atomically.** `POST /auth/activation/complete { token, otp }` verifies the OTP against the pending secret, then in one transaction:
   - `UPDATE … SET used_at = now() WHERE token_hash = ? AND used_at IS NULL` (0 rows → abort)
   - save the password hash, create the `TotpCredential` row, save hashed recovery codes

Then delete the Redis pending setup (if that fails, its TTL cleans up), create a fresh session, show the recovery codes once.

As built (`InvitationService`)

- The client keeps the raw invite token in memory from the `#token=` fragment and sends it with each request; the server always hashes it itself. Never return the raw token or its hash.
- Step 2 (`setNewPassword`) returns only `{ otpauthUri }` (plus the raw secret if manual entry is offered). The URI is built from the plain secret with issuer `Dentacore` and label `sentToEmail` (a snapshot of the address the invite went to).
- The pending payload is `{ employeeId, passwordHash, encryptedSecret }`, TTL = `min(15 min, token's remaining lifetime)`; a TTL ⩽ 0 is rejected as an expired token.
- Step 3 (`verifyTotp`) takes only `{ inviteToken, otp }` and reads everything else from Redis. Order: decrypt → verify → transaction (conditional token consume first, then create `MfaCredentials` with `lastUsedTimeStep` = the activation code's `timeStep`, then set the password and `ACTIVE` status with a conditional update) → best-effort `DEL` of the pending payload after commit (`.catch(log)`).
- A failed Redis `SET` in step 2 propagates (the user retries); the final `DEL` is cleanup after success, so it must not turn success into an error. No outbox for either.
- Plan: sign the user in at the end of activation, so an immediate login never needs to reuse the activation code.
- Attempt limit on activation codes: on hold (anyone with the token can redo step 2 anyway).

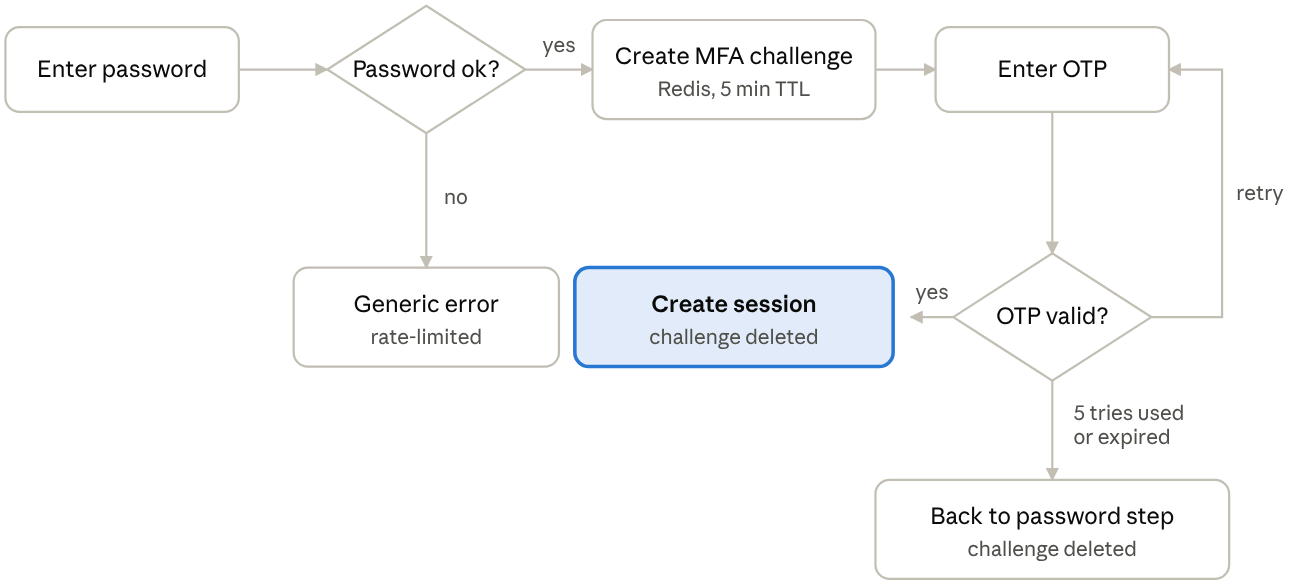
Token in the URL

- Remove it from the address bar with `history.replaceState` after reading it.
- `Referrer-Policy: no-referrer` and no third-party scripts on the activation page.
- Don't log full URLs containing the token.

2FA login flow

A correct password creates only a short-lived MFA challenge in Redis; the real session is created after a valid OTP.

A session exists only after a valid OTP



2FA login flow · 2 decisions, 1 retry loop

A wrong OTP retries within the same challenge; after 5 tries or 5 minutes the user starts again from the password.

Policy: 2FA is mandatory for staff, enforced as clinic policy rather than a personal setting. (Original idea: on by default, can be turned off in settings.) If a toggle is ever kept, turning it off requires password + current OTP and sends an email notification. Optional 2FA is reasonable for a future patient portal.

Server-side OTP timeout (replaces the hidden form timeout)

- A client-side hidden field or JS timer enforces nothing.
- Correct password → create `mfa:{randomId}` → `{ userId, attempts }` in Redis, TTL ~5 min; give the ID to the client in a short-lived cookie.
- OTP endpoint: challenge missing/expired → back to the password step; OTP valid → delete the challenge, create the session.

Attempt limits

- ~5 attempts per challenge, then delete it.
- Per-account backoff/lockout across challenges, so re-entering the password doesn't grant fresh attempts.

TOTP details

- Accept ±1 time step (30 s) for clock drift.
- Reject reused codes: store `lastUsedStep` per user.
- Use `otplib`; don't implement TOTP by hand.
- Recovery codes (about 10, single-use, shown once) cover a lost phone.

As built (`AuthService`)

Login uses the **employee number** (unique, never changes; included in the invite email). Email stays the recovery channel.

1. **`login`** (password step): look up the active employee by number. Unknown number and wrong password both return `401 "Invalid employee number or password"`. On success, create `mfa:{sha256(challengeId)}` → `{ employeeId, rememberMe }` with a 5-min TTL and return the raw challenge ID. No session ID is created here.
2. **`verifyOtp`**: `GET` the challenge → load the employee by ID (roles, `mfaCredentials` with `encryptedSecret` and `lastUsedTimeStep`; must still be active) → decrypt → `verify` with `afterTimeStep` → claim the time step with a conditional update → **claim the challenge**: `DEL` it and require a result of 1 → generate the session ID → `MULTI` (session, set, NX/GT TTLs) → prune the set → return `{ sessionIdRaw, rememberMe }`.

All errors in `verifyOtp` are a generic `401 "Invalid credentials"`. The challenge is claimed only after a correct code, so a wrong code can be retried; claiming before creating the session means two simultaneous requests can't both get a session.

Replay protection

- `MfaCredentials.lastUsedTimeStep Int?`. otplib's `verify` takes `afterTimeStep` (rejects that step and earlier) and returns the matched `timeStep` on success.
- `afterTimeStep` alone has a race, so the step is also recorded with `updateMany ... WHERE lastUsedTimeStep IS NULL OR lastUsedTimeStep < :step` and `count === 1` (the otplib guide suggests a row lock; the conditional update gives the same result in Prisma).
- `epochTolerance: 30` (otplib's default is 0, current period only). Log non-zero `delta` to measure real clock drift; tighten later if drift is rare. Keep the server clock NTP-synced.
- Top-level `verify` returns a TOTP|HOTP union; narrow with `'timeStep' in result`, and keep the result as one object rather than destructuring it.
- Side effect: two sign-ins within the same 30 s need the next code.

- This is **replay** protection, not phishing protection: a real-time phishing proxy uses the code first. Phishing resistance needs passkeys/WebAuthn.

Attempt limits (required for login, not yet built)

- Per challenge: `INCR` + `EXPIRE NX` before verifying; more than 5 → delete the challenge.
- Per account: `mfa:fail:{employeeId}`, 10 failures per 15 min → refuse all codes until the window passes, email the employee, reset on success.
- Per IP: `@nestjs/throttler` on auth endpoints (Redis storage once there are multiple instances).

Storage: tokens, secrets, recovery codes

Invite tokens are stored as SHA-256 hashes; TOTP secrets are encrypted with XChaCha20-Poly1305 (`@noble/ciphers`) and stored in Postgres via Prisma; recovery codes will be argon2id hashed.

Item	Where	Protection	Why
Invite token	Email link `#token=` (raw), `InviteToken.tokenHash`	SHA-256	256-bit random: no brute force; must be looked up by hash
TOTP secret	Redis pending payload → `MfaCredentials.encryptedSecret`, moved unchanged	XChaCha20-Poly1305, AAD = employee ID	Server needs the original to compute codes
`TOTP_ENC_KEY`	Environment only (validated: base64, 32 bytes)	Never in the DB	A DB leak must not include the key
Password hash	Redis pending payload → `Employee.passwordHash`	bcrypt	Committed only at activation

Item	Where	Protection	Why
Session ID	Cookie (raw), Redis key `session:{sha256}`	SHA-256	A Redis dump yields no usable cookies
Recovery codes (planned)	Postgres, hashed	argon2id	Shorter codes need slow hashing

Schema

```
model Employee {
  id             String  @id @default(uuid())
  employeeNumber String  @unique          // login name
  passwordHash   String?                  // null until activation
  inviteStatus   InviteStatus             // UNINVITED | INVITED
  activeStatus   ActiveStatus             // INACTIVE | ACTIVE | DEACTIVATED
  deletedAt      DateTime?
  // ...
}

model InviteToken {
  tokenHash   String    @unique           // sha256(rawToken)
  employeeId  String
  sentToEmail String                      // TOTP label
  expiresAt   DateTime
  consumedAt  DateTime?
  revokedAt   DateTime?
  // ...
}

model MfaCredentials {
  encryptedSecret  String                 // base64(nonce + ciphertext + tag)
  lastUsedTimeStep Int?                   // replay protection
  employeeId       String  @unique
  // ...
}
```

The employee row exists before the invite (`UNINVITED`, no password), so the token always points at a real employee. Consider merging the two status fields into one lifecycle enum (`UNINVITED → INVITED → ACTIVE → DEACTIVATED`) so a deactivated employee can never look like a pending invite.

Encryption (`SecretCipherService`)

- `managedNonce(xchacha20poly1305)` generates a fresh 24-byte nonce per encryption and stores it in the output, so there is no IV to manage. Random nonces are safe at this size.
- AAD = employee ID: never encrypted or stored, supplied again at every decryption. It binds a secret to its owner, so a ciphertext copied into another employee's row fails to decrypt. It must be stable and available without trusting the ciphertext (an email would not qualify).
- Encrypt only the TOTP secret, never the whole pending payload.
- The key is read through `ConfigService` in the constructor and decoded from base64. A decrypt failure throws and should surface as a logged 500, never as "wrong code".
- `node:crypto` AES-256-GCM is an equivalent alternative (12-byte IV per encryption, stored with the auth tag in `v1:<iv>:<tag>:<ct>`). Never a fixed or module-level IV, and never CBC (no tamper detection, no AAD).

```
encrypt(plain: string, aad: string): string {
  const box = managedNonce(xchacha20poly1305)(this.key, utf8ToBytes(aad));
  return Buffer.from(box.encrypt(utf8ToBytes(plain))).toString('base64');
}

decrypt(stored: string, aad: string): string {
  const box = managedNonce(xchacha20poly1305)(this.key, utf8ToBytes(aad));
  return bytesToUtf8(box.decrypt(Buffer.from(stored, 'base64'))); // throws
on any mismatch
}
```

NestJS implementation notes

These are the code-level conventions the auth work follows.

Request user and module boundaries

- The global guard reads Redis once per request and sets `request.user = { userId, role, clinicId, sessionId }`; `@CurrentUser()` (a `createParamDecorator`) reads it. Services take values as arguments, not the request. No request-scoped providers; add `nestjs-cls` only for deep code such as audit logging or `clinicId` query scoping.
- A service uses its own module's repository and other modules' services, never another module's repository. `@nestjs-cls/transactional` lets service calls share one transaction.
- `SessionStore` (`src/auth/stores/session.store.ts`) is the only code that writes sessions: `create` (calls private `pruneExpired`), later `getAndTouch`, `delete`, `deleteAllForUser`. `AuthService` decides; the store executes.

Cookies

- The cookie holds the **raw** session ID; only the server hashes it. The controller sets it with `@Res({ passthrough: true })` so interceptors still run. Never return the ID in the body.
- Options live in one place: `__Host-sid`, `httpOnly`, `secure`, `sameSite: 'lax'`, `path: '/'`; reuse them for `clearCookie`. `maxAge` only for remember-me.
- `cookieParser()` without a signing secret. `__Host-` needs `Secure`, which browsers allow on `http://localhost` only.
- Credentials never go in URLs (path or query): the challenge ID goes in an HttpOnly cookie or the body, the OTP in the body.

Configuration

- Nest classes read config through `ConfigService.getOrThrow` in the constructor; `main.ts` and `prisma7.config.ts` keep `process.env`.
- `validateEnv` checks every variable the app reads (URLs, ports, `SMTP_SECURE`, a 32-byte base64 `TOTP_ENC_KEY`) and reports all problems at once.
- `ApiKeyMiddleware` compares with `timingSafeEqual` after a length check. Open question: it runs on every route, which puts the key in the browser.

Patterns

- **Conditional update (compare-and-set):** every once-only transition puts its condition in the write and checks the count: consuming invite tokens, `markInvited`, activation, `lastUsedTimeStep`. In Redis the same idea is claiming a challenge with `DEL` == 1.

Prisma queries are lazy: always `await` them. Name such methods as writes (`consume`, `claim`), not reads.

- **Error handling:** catch errors that map to a meaningful response (P2002 → 409) and rethrow the rest; let infrastructure failures (Redis or the database down) propagate; best-effort cleanup after a success catches and logs. An outbox is only for side effects that must follow a committed database write (the invite email).
- **ioredis:** `MULTI` and pipeline commands only queue until `exec()`, which resolves to `[err, result]` pairs in queue order and doesn't reject for a single failed command. `SREM` with zero members is an error.
- Login lookup uses the employee number once at the edge; everything after uses the employee UUID. Roles are read from the database when the session is created, never from the client.

Open items

- [ ] Login attempt limits: per challenge (5) and per account (10 per 15 min, with an alert email). Required before real use.
- [ ] Dummy-hash comparison in `login` for unknown employee numbers (timing currently reveals which numbers exist).
- [ ] Null-check the `mfaCredentials` relation itself (optional chaining) before reading `encryptedSecret`.
- [ ] Activation: store `timeStep` when creating `MfaCredentials` rather than calling the time-step claim before the row exists; sign the user in at the end of activation.
- [ ] `AuthModule` must import `MfaCredentialsModule`; select `lastUsedTimeStep` in `findActiveEmployeeById`; don't destructure the verify result.
- [ ] Controllers: activation and login endpoints; set and clear cookies; fill `userAgent` and `ip`.
- [ ] Move session creation and pruning into `SessionStore`; check `exec()` results.
- [ ] Sync `package-lock.json` (`npm ci` fails).
- [ ] Design TOTP secret regeneration (use otplib's staged `replace` flow as reference; active secret stays until the new one is verified).
- [ ] Recovery codes (10, single-use, argon2id).
- [ ] Decide on one employee lifecycle status field.
- [ ] Load-test the session guard (k6) and Redis (`redis-benchmark`).

- [ ] Later: passkeys/WebAuthn for phishing resistance.